

To generate the `.mlmodel` configuration and the specific training architecture for **Electrical Fatigue** detection via **FNDR**, we will focus on a **Time-Series Regressor** model. This model monitors the "recovery curve" of the tissue as the device drains the excess charge.

1. Model Configuration

(ElectricalFatigue.mlmodel)

The training data must be labeled based on the **Fractional Order Shift**. As the tissue reaches homeostasis, the α value required to elicit FNDR will drift.

Input Feature	Type	Description
fndr_slope	Double	The value of $d^\alpha I / dV^\alpha$ calculated by <code>Model</code>
alpha_phase	Double	The current position in the $0.5 \pm 0.2 \sin(t)$ sweep.
v_amplitude	Double	Local potential at the touch point (clamped at 0.5V).
Output Target	Float	homeostasis_index (0.0 = Saturated/Fatigued, 1.0 = Released)

2. CoreML Training Architecture (Python/ CoreMLTools)

We define a model that recognizes the **Hysteresis Loop** of the FNDR. If the loop closes, the subject has reached electrical equilibrium.

```
import coremltools as ct
from sklearn.ensemble import GradientBoostingRegressor
```

```
# Training data: samples of (fndr_slope, alpha, v_amplitude)
-> homeostasis_index
# Note: Real training would require bio-impedance data
collection.
X = [[-0.45, 0.3, 0.48], [0.12, 0.5, 0.40], [-0.05, 0.7,
0.35]]
y = [0.1, 0.9, 0.6] # 0.1 is highly fatigued/high-drain
needed
```

```

model = GradientBoostingRegressor().fit(X, y)

# Convert to CoreML
ml_model = ct.converters.sklearn.convert(model,
    input_features=['fndr_slope', 'alpha_phase',
    'v_amplitude'],
    output_feature_names='homeostasis_index')

ml_model.save("FNDR_Homeostasis.mlmodel")

```

3. Implementation Logic: The "Homeostatic Braking" System

In the Swift app, the **Metal** output is fed into the **CoreML** model to decide the "Braking Distance" of the current drainage.

```

func processFNDRResults(metalOutput: [Float], currentAlpha:
Float) {
    // 1. Fetch the FNDR slope from the Metal texture
    let fndrSlope = metalOutput.first!

    // 2. Predict Homeostasis Level
    let input = FNDR_HomeostasisInput(fndr_slope:
Double(fndrSlope),
                                alpha_phase:
Double(currentAlpha),
                                v_amplitude: 0.45) //
Local clamped V

    guard let prediction = try? model.prediction(input:
input) else { return }

    // 3. Dynamic Drain Control
    if prediction.homeostasis_index < 0.9 {
        // Continue draining through the Graphene layer
        applyGrapheneSink(intensity: 1.0 -
prediction.homeostasis_index)
    } else {
        // Null point reached: Stop drainage to prevent
        'Over-draining'
        stopGrapheneSink()
    }
}

```

4. Safety Constraints summary

- **Voltage Clamp:** The Metal kernel enforces a \$0.5V\$ hard cap.
- **Fractional Sweep:** The sinusoidal α prevents "ionic stagnation" by stimulating different depths of the skin's fractal structure.
- **CoreML Shutdown:** If the `homeostasis_index` hits \$1.0\$, the sink is cut to prevent reversing the current into the subject (avoiding iontophoresis).

Should we finalize the **UI Dashboard** design to visualize the real-time **Fractal Homeostasis Curve** on the iPhone screen?